

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación

Construcción de Compiladores
Proyecto 2

Análisis Semántico de Compiscript

Documentación de arquitectura y ejecución

Compiler Toolkit

Estudiante

Osman Emanuel de León García
Carné 23428

Guatemala, agosto de 2026

1. Introducción y alcance

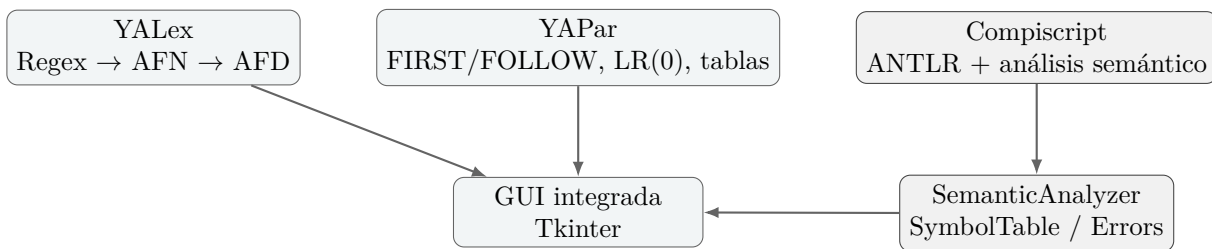
Compiler Toolkit reúne las tres etapas desarrolladas a lo largo del proyecto del curso. En las primeras entregas se construyeron YALex, un generador de analizadores léxicos, y YAPar, un generador de analizadores sintácticos. La etapa final se concentra en Compiscript, donde se integra ANTLR4 para el análisis léxico y sintáctico y se implementa un analizador semántico propio sobre el árbol producido por el parser.

El objetivo de esta documentación es presentar la arquitectura de la implementación y dejar un procedimiento claro para instalar y ejecutar el proyecto. Por esa razón, YALex y YAPar se describen como antecedentes técnicos y el mayor detalle se dedica a Compiscript, su tabla de símbolos, manejo de ámbitos, validaciones semánticas e integración con la interfaz gráfica.

El alcance corresponde al *front-end* de un compilador: reconocimiento léxico, análisis sintáctico y validación semántica. No se incluye generación de código intermedio, optimización ni generación de código objetivo.

2. Arquitectura general

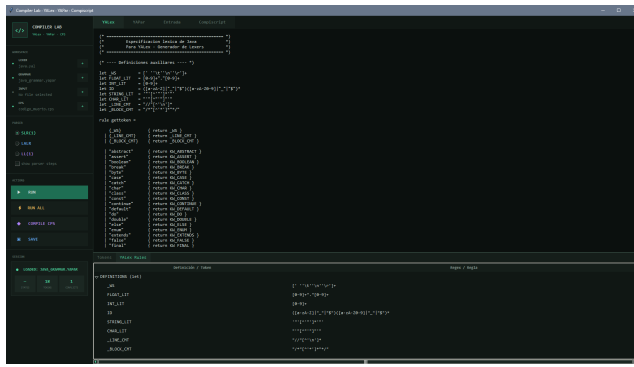
La solución se organizó en módulos con responsabilidades separadas. Esto permite ejecutar cada herramienta desde línea de comandos y, al mismo tiempo, reutilizar sus resultados desde la interfaz gráfica.



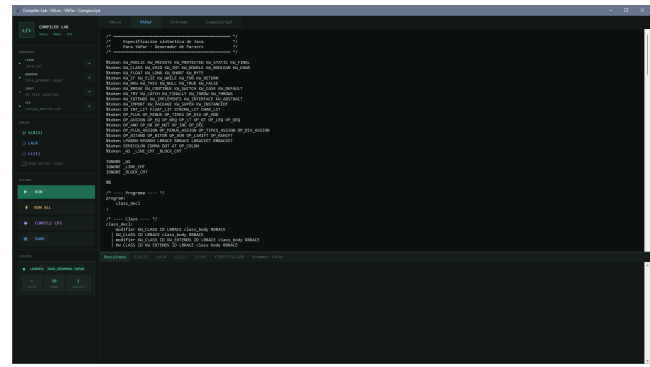
2.1. Etapas anteriores: YALex y YAPar

YALex recibe una especificación `.yal`. Las expresiones regulares se convierten a árboles, luego a AFN mediante la construcción de Thompson y finalmente a AFD mediante construcción de subconjuntos. El generador produce un lexer ejecutable en Python y aplica *longest match* junto con prioridad por orden de reglas.

YAPar recibe una gramática `.yapar`. A partir de las producciones calcula FIRST y FOLLOW, construye el autómata LR(0) y genera tablas SLR(1), LALR o LL(1). Los métodos SLR/LALR utilizan un motor *shift-reduce*, mientras que LL(1) utiliza un motor predictivo.



(a) Especificación YALex dentro de la GUI.

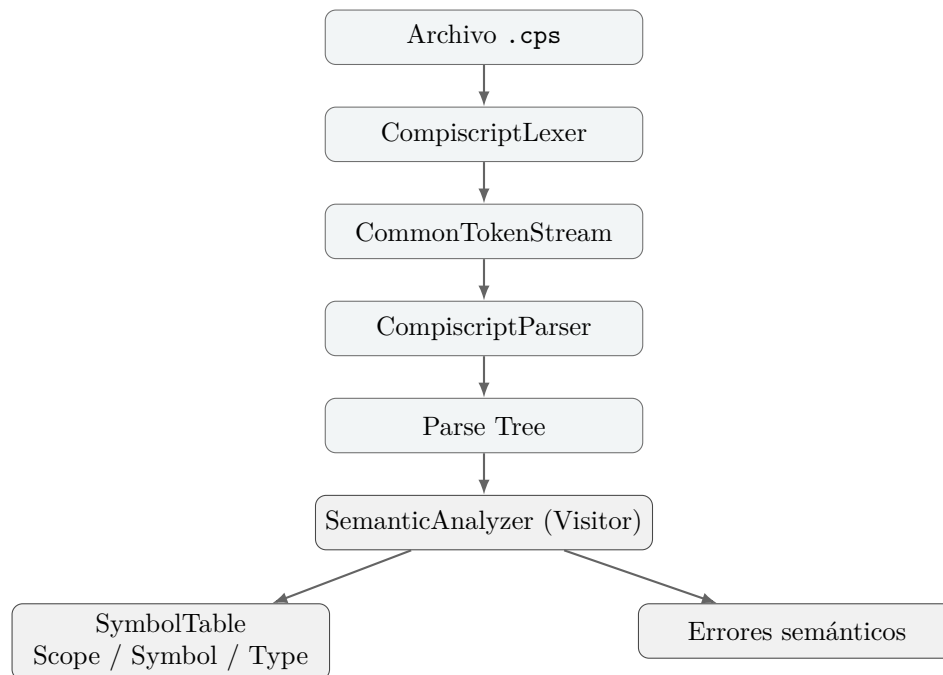


(b) Gramática YAPar dentro de la GUI.

Figura 1: Etapas léxica y sintáctica desarrolladas previamente.

3. Arquitectura de Compiscript

Compiscript es un subconjunto de TypeScript. Su gramática se encuentra en `grammars/compiscript/Compiscript.g4`. ANTLR 4.13.2 genera el lexer, parser y Visitor base; sobre este último se implementa `SemanticAnalyzer`. El uso del patrón Visitor permite separar el recorrido del árbol de la gramática generada, una práctica habitual al extender parsers de ANTLR con procesamiento propio [2, 3].

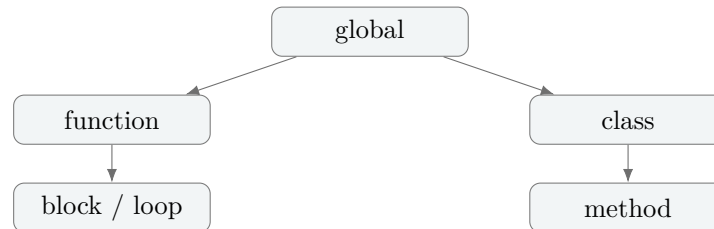


Los archivos principales de esta etapa son:

- `Compiscript.g4`: define las reglas léxicas y sintácticas del lenguaje.
- `compiscript/generated/`: contiene el lexer, parser y Visitor generados por ANTLR.
- `semantic.py`: implementa el recorrido semántico del árbol.
- `symbol_table.py`: define tipos, símbolos, ámbitos y la tabla de símbolos.
- `errors.py`: concentra las clases de errores semánticos.
- `compiscript.py`: ofrece la ejecución desde línea de comandos.

3.1. Análisis semántico y ámbitos

El analizador semántico visita las construcciones reconocidas por el parser y mantiene el contexto mediante ámbitos anidados. Cada scope conserva una referencia a su padre, por lo que la búsqueda de un identificador comienza en el ámbito actual y asciende hasta el global. Este mecanismo permite representar variables locales, parámetros, funciones anidadas, métodos y bloques sin mezclar sus declaraciones.



La implementación semántica se organizó alrededor de las categorías solicitadas para la entrega. En el **sistema de tipos** se revisan asignaciones, operaciones aritméticas y lógicas, comparaciones, listas e índices. En **manejo de ámbitos** se controla la resolución de nombres, la visibilidad y la redeclaración dentro de un mismo entorno. Para **funciones** se comprueban parámetros, llamadas, retornos, recursividad y funciones anidadas. El **control de flujo** valida condiciones y el contexto de instrucciones como **break**, **continue** y **return**. Finalmente, el soporte de **clases y objetos** contempla miembros, constructores, herencia y **this**. También se detectan situaciones como código inalcanzable después de determinadas instrucciones de transferencia.

Estas verificaciones se apoyan en la tabla de símbolos y en el contexto acumulado durante el recorrido del árbol. La separación entre análisis sintáctico y semántico sigue la organización clásica del front-end de un compilador, donde la estructura reconocida por el parser se complementa con información de tipos, declaraciones y ámbitos [1].

SÍMBOLO / SCOPE	CLASE	TIPO	ÁMBITO	LÍNEA	DETALLES
global	SCOPE - GLOBAL		global	7 símbolo(s)	
f print	FUNCIÓN	void	global	-	(value:any) -> void
integer	CLASE	integer	global	-	class
string	CLASE	string	global	-	class
boolean	CLASE	boolean	global	-	class
Persona	CLASE	Persona	global	1	class
p	VARIABLE	Persona	global	7	

Figura 2: Tabla de símbolos mostrada por la interfaz. La vista conserva el ámbito, clase de símbolo, tipo y detalles asociados.

4. Interfaz gráfica e inspección de resultados

La interfaz se implementó en `gui/app.py` con Tkinter y funciona como un pequeño entorno de trabajo para los tres componentes. En Compiscript integra editor de código, tokens, árbol sintáctico, errores por fase, tabla de símbolos, referencia del lenguaje y gramática. El análisis se ejecuta en un hilo separado para evitar que la ventana deje de responder durante el procesamiento.



```

graph TD
    program[program] --> statement1[statement]
    statement1 --> classDeclaration[classDeclaration]
    statement1 --> variableDeclaration1[variableDeclaration]
    statement1 --> statement2[statement]
    classDeclaration --> className[className]
    classDeclaration --> block[block]
    className --> saludar["saludar"]
    block --> functionDeclaration[functionDeclaration]
    functionDeclaration --> functionName[functionName]
    functionDeclaration --> parameters[parameters]
    functionDeclaration --> body[block]
    functionName --> saludar2["saludar"]
    parameters --> emptyParams["()"]
    body --> statement3[statement]
    statement3 --> expressionStatement1[expressionStatement]
    statement3 --> returnStatement1[returnStatement]
    expressionStatement1 --> expression1[expression]
    expression1 --> Person1["Person()"]
    returnStatement1 --> expression2[expression]
    expression2 --> Person2["Person()"]
    variableDeclaration1 --> variableName[variableName]
    variableDeclaration1 --> typeAnnotation[typeAnnotation]
    variableName --> p["p"]
    typeAnnotation --> type[type]
    type --> Type["Type"]
    variableDeclaration1 --> initializer1[initializer]
    initializer1 --> Person3["Person()"]
    statement2 --> expressionStatement2[expressionStatement]
    statement2 --> returnStatement2[returnStatement]
    expressionStatement2 --> expression3[expression]
    expression3 --> Person4["Person()"]
    returnStatement2 --> expression4[expression]
    expression4 --> Person5["Person()"]
    variableDeclaration2[variableDeclaration] --> variableName2[variableName]
    variableDeclaration2 --> typeAnnotation2[typeAnnotation]
    variableName2 --> x["x"]
    typeAnnotation2 --> type2[type]
    type2 --> Type2["Type"]
    variableDeclaration2 --> initializer2[initializer]
    initializer2 --> Person6["Person()"]
  
```

Los errores se separan por fase. En la Figura siguiente se observa un caso de código muerto: la declaración posterior a **return** es reconocida sintácticamente, pero el analizador semántico la reporta como una instrucción inalcanzable.

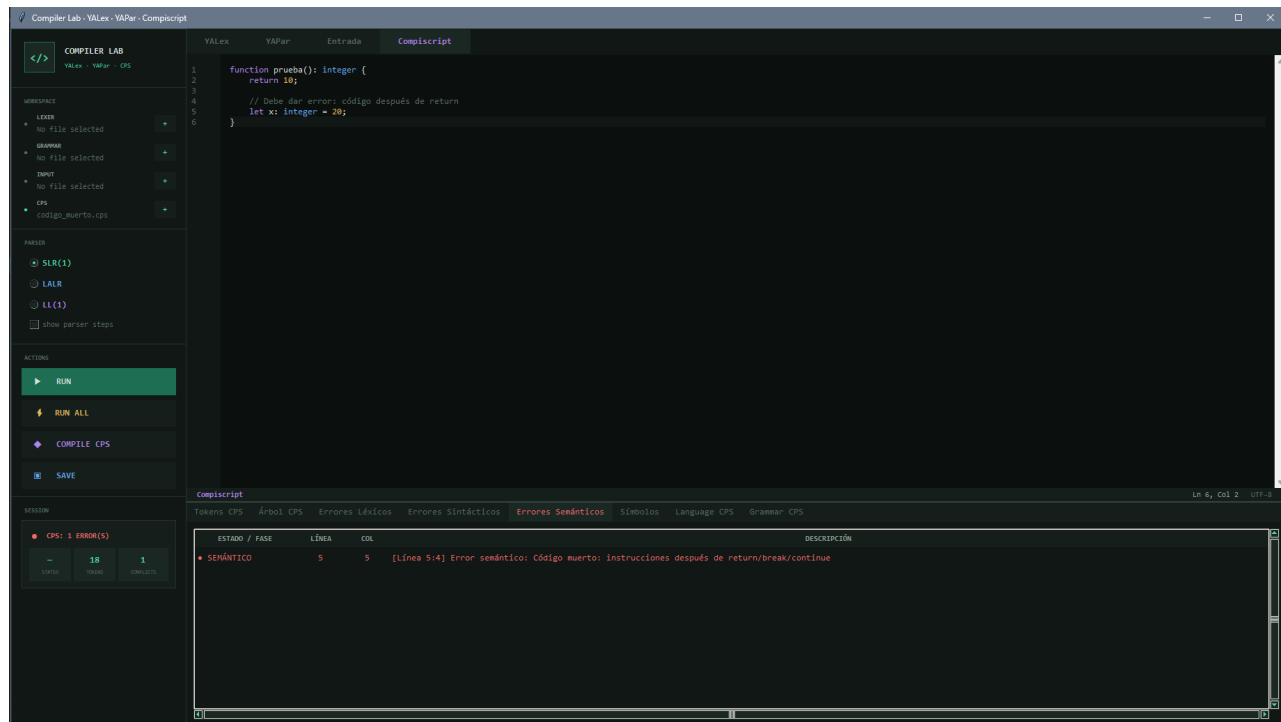


Figura 5: Ejemplo de error semántico detectado y presentado en la GUI.

5. Instalación y ejecución

Los comandos siguientes se ejecutan desde la raíz del repositorio. El entorno utilizado para el desarrollo es Python 3.11 sobre Windows.

5.1. Requisitos

- Python 3.11.
- Java Runtime, necesario para ejecutar el generador de ANTLR.
- Graphviz instalado en el sistema y disponible en PATH [4].
- Git para clonar el repositorio.
- Tkinter, incluido normalmente con la instalación estándar de Python en Windows.

5.2. Preparación del entorno

```

git clone https://github.com/Emadlbg/compiler-toolkit.git
cd compiler-toolkit

python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

```

Las dependencias Python fijadas por el proyecto son `antlr4-python3-runtime==4.13.2` y `graphviz==0.21`. Para comprobar las herramientas externas pueden utilizarse:

```
java --version
```

```
dot -V
java -jar tools\antlr.jar
```

La última instrucción debe indicar ANTLR Parser Generator Version 4.13.2.

5.3. Ejecución de la interfaz

La forma más directa de utilizar el proyecto completo es:

```
python gui/app.py
```

Desde la GUI se pueden cargar las especificaciones YALex/YAPar, entradas de prueba y archivos .cps.

5.4. Ejecución desde línea de comandos

Compiscript

```
python compiscript.py tests\compiscript\tipos\ok_tipos.cps --all
```

La opción `-all` muestra árbol sintáctico, resultado semántico y tabla de símbolos.

YALex

```
python yalex.py grammars\java\java.yal -o java_lexer.py --all
python java_lexer.py examples\java\input_java.txt --verbose
```

YAPar

```
python yapar.py grammars\java\java_grammar.yapar \
-l grammars\java\java.yal \
-i examples\java\input_java.txt --all
```

5.5. Regeneración de ANTLR

Los archivos generados ya están incluidos en el repositorio, por lo que este paso no es necesario para ejecutar Compiscript. Solo debe realizarse después de modificar `Compiscript.g4`:

```
java -jar tools\antlr.jar -Dlanguage=Python3 -visitor \
-Xexact-output-dir -o compiscript\generated \
grammars\compiscript\Compiscript.g4
```

La opción `-Xexact-output-dir` mantiene los archivos generados directamente dentro de `compiscript/generated/`, que es la ubicación esperada por la aplicación.

6. Batería de pruebas y cierre

Compiscript cuenta con una batería de casos de prueba funcionales organizada por tipos, ámbitos, funciones, control de flujo, clases y casos adicionales. Cada categoría incluye programas exitosos y programas diseñados para provocar errores concretos, de acuerdo con el enfoque solicitado en la

especificación del proyecto. Esto permite comprobar tanto el recorrido normal como las validaciones semánticas sin depender únicamente de ejemplos escritos durante la ejecución.

Grupo	Aspectos cubiertos
tipos/	Declaraciones, asignaciones y compatibilidad de tipos.
ambitos/	Visibilidad, redeclaraciones y resolución de identificadores.
funciones/	Parámetros, llamadas, retornos y funciones anidadas.
control_flujo/	Condiciones, ciclos e instrucciones de transferencia.
clases/	Clases, métodos, atributos, constructores y herencia.
extra/	Constantes, closures, código muerto, listas e índices.

En conjunto, el proyecto conecta los conceptos trabajados durante el curso con una implementación ejecutable. YALex y YAPar permiten observar la construcción de analizadores desde sus algoritmos base; Compiscript completa el flujo con una etapa semántica que incorpora contexto, tipos y ámbitos. La GUI no sustituye estas implementaciones, sino que funciona como una capa de integración para inspeccionar sus resultados y facilitar las pruebas.

Referencias

- [1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.a ed. Pearson, 2006.
- [2] T. Parr, *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2013.
- [3] ANTLR Project, “ANTLR Documentation.” Disponible en: <https://www.antlr.org/>. Consultado en agosto de 2026.
- [4] Graphviz, “Graphviz Documentation.” Disponible en: <https://graphviz.org/documentation/>. Consultado en agosto de 2026.